

DEVELOPER HANDBOOK · VOL. 1

Spec-Driven Development

with Claude Code

Vibe Coding · TDD · BDD · DDD · SDD · Hooks · Agents · Sub-agents · MCP · Skills · Memory · Traceability · Observability

Author: Jampala Naresh

Software Engineer & Developer Advocate
Specialist in AI-Assisted Development Workflows & Claude Code

First Edition · 2025

Ch 01	Ch 02	Ch 03	Ch 04	Ch 05	Ch 06
Vibe Coding	TDD	BDD	DDD	SDD Principles	Terminology
Ch 07	Ch 08	Ch 09	Ch 10	Ch 11	Ch 12
Architecture	Hooks	Agents	Skills	Memory	MCP
Ch 13	Ch 14	Ch 15	Ch 16	Ch 17	
4 Phases	Full Example	Folder Structure	Traceability	Quick Ref	

Table of Contents

Ch	Title	Key Content
01	Vibe Coding	Definition · Pros · Cons · Loop diagram · When to use
02	TDD — Test-Driven Development	Red-Green-Refactor · Cycle diagram · Full JS example
03	BDD — Behaviour-Driven Development	Given/When/Then · Gherkin · Cucumber JS example
04	DDD — Domain-Driven Design	Ubiquitous language · Bounded contexts · Full JS example
05	SDD — Spec-Driven Development	Definition · 5 principles · Pipeline diagram · vs TDD/BDD/DDD
06	Core Terminology A–Z (30+ terms)	Colour-coded cards: Agent · CLAUDE.md · Hook · MCP · Skill...
07	Claude Code Architecture	All components · Session lifecycle · Context window tips
08	Hooks — Complete Reference	4 events · Lifecycle diagram · Config JSON · Design patterns
09	Agents, Sub-agents & Orchestration	Orchestration diagram · Roles · Coherence rules · Prompts
10	Skills — Types, Anatomy & Creation	Routing diagram · SKILL.md anatomy · All public skills
11	Memory — All 4 Layers	Pyramid diagram · What to store · Multi-file pattern
12	MCP — Model Context Protocol	Architecture diagram · 6 servers · Config · SDD integration
13	The 4 SDD Phases — Deep Dive	Per-phase guide · CLAUDE.md template · Gate checklists
14	Full Tasks API Example	Complete CLAUDE.md · All 8 source files · 9 tests · Results
15	Folder Structure	Terminal tree · All files annotated · File creation order
16	Traceability & Observability	Chain diagram · 5 obs. layers · Full traceability matrix
17	Comparison Tables & Quick Reference	Master table · CLI prompts · Gates · Anti-patterns · Index

About the Author: Jampala Naresh is a Software Engineer and Developer Advocate specialising in AI-assisted development workflows. This handbook consolidates hands-on experience building production systems using Claude Code and Spec-Driven Development into a ~50-page practitioner reference.

Ch 01

Vibe Coding

Freestyle prompt-driven development — full definition, anatomy & limits

Definition

Vibe coding is a freestyle, prompt-driven approach to software development where the developer describes intent in natural language, an AI agent writes or edits code in response, and the developer reviews or rejects each edit in real time. There is no upfront specification — requirements exist only in the developer's head and accumulate incrementally through a series of conversational prompts. The name captures the improvisational, intuitive feel of the workflow.

The Vibe Coding Loop

```
1. Developer types: 'Build me an API that manages tasks' 2. AI writes app.js ← developer reviews → accept 3. Developer types: 'Add a status field to tasks' 4. AI edits app.js ← developer reviews → accept 5. Developer types: 'Make it return 404 when a task is not found' 6. AI edits again ← developer reviews → repeat indefinitely
```

Strengths of Vibe Coding

- Fastest path from idea to running code — ideal for single-file scripts and throwaway tools.
- Zero planning overhead — start coding immediately, iterate freely without documentation.
- Excellent for exploration — quickly validate whether an idea is technically feasible.
- Superb learning tool — work alongside Claude to understand a new library or API pattern.
- Low ceremony — no planning documents, review gates, or structured approval processes.

Limitations at Scale

- Context loss — Claude has no persistent memory between sessions without CLAUDE.md.
- Review fatigue — reviewing every micro-edit is exhausting on multi-file features.
- Spec drift — requirements live only in the developer's head, never in a versioned document.
- Architecture incoherence — successive prompts can produce contradictory design choices across files.
- Untraceable tests — tests validate code that already exists, not pre-defined requirements.
- High 'that is not what I meant' rate — ambiguity rises sharply as feature scope grows.

When to Use Vibe Coding

- Single-file scripts, throwaway automations, one-off data transformations.
- Hackathon demos where speed matters more than maintainability or correctness.
- Technical spikes — prototyping to test feasibility before writing a proper spec.
- Learning sessions — working alongside Claude to understand an unfamiliar library.

When NOT to Use Vibe Coding

- Any feature touching more than one file or requiring cross-module coordination.
- Team projects where other developers need to read, understand, and extend the code.

- Production APIs with defined requirements that must be traceable and auditable.
- Long-lived codebases that will be maintained or extended over months or years.

Rule of thumb: if your feature touches more than one file, or will be maintained by more than one person, switch to Spec-Driven Development from the start.

Ch 02

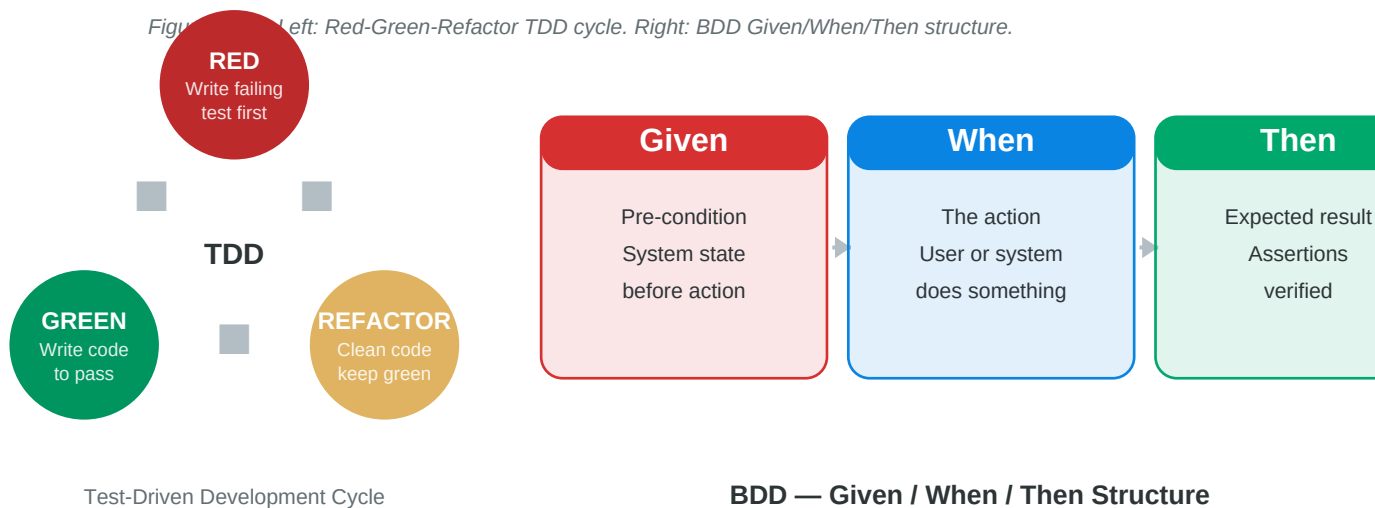
TDD — Test-Driven Development

Red-Green-Refactor · Cycle diagram · Full code example

Definition

Test-Driven Development (TDD) is a software development methodology in which automated tests are written *before* the production code they validate. The developer writes a failing test first, then writes the minimum code needed to make the test pass, then refactors the code for quality — keeping all tests green throughout. This is the famous **Red** → **Green** → **Refactor** cycle, coined by Kent Beck in his book *Test-Driven Development: By Example* (2002).

TDD Cycle Diagram & BDD Given/When/Then



The Three Phases Explained

- **RED** — Write a test for a feature that does not exist yet. The test **MUST** fail because the production code has not been written. A test that immediately passes means the test is wrong or the feature already exists. Red = clear signal that the test is meaningful.
- **GREEN** — Write the absolute minimum production code to make the failing test pass. Do not optimise, do not add extra features, do not gold-plate. Just make it green as fast as possible. Ugly code is fine here — refactoring comes next.
- **REFACTOR** — Clean up the code — extract functions, improve naming, remove duplication, apply design patterns — while keeping all tests green. Refactoring is safe because the tests are a comprehensive safety net. If anything breaks, you know immediately.

TDD — Complete Code Example (Tasks API)

Step 1 — RED: Write the failing test first

```
// tests/task.model.test.js (written BEFORE models/task.js exists) const Task =
require('../models/task'); // file does not exist yet! test('create() returns a task with
status todo', () => { const task = Task.create('Buy milk'); expect(task.title).toBe('Buy
```

```
milk'); expect(task.status).toBe('todo'); // FAILS - Task.create does not exist
expect(task.id).toBeDefined(); // FAILS expect(task.createdAt).toBeDefined(); // FAILS });
test('create() rejects empty title', () => { expect(() => Task.create('')).toThrow('title
required'); // FAILS });
```

Step 2 — GREEN: Write minimum code to pass

```
// models/task.js - written to make the failing tests pass
const { v4: uuidv4 } = require('uuid');
const Task = {
  create: (title) => {
    if (!title) throw new Error('title required');
    // makes test 2 pass
    return {
      id: uuidv4(), // makes id assertion pass
      title, // makes title assertion pass
      status: 'todo', // makes status assertion pass
      createdAt: new Date().toISOString() // makes createdAt assertion pass
    };
  }
};
module.exports = Task;
// Run: npm test -> BOTH TESTS GREEN ✓
```

Step 3 — REFACTOR: Improve without breaking

```
// models/task.js - refactored: add persistence, keep tests green
const { v4: uuidv4 } = require('uuid');
const db = require('../db/schema');
const Task = {
  create: (title) => {
    if (!title || typeof title !== 'string') throw new Error('title required');
    if (title.length > 200) throw new Error('title too long');
    const task = {
      id: uuidv4(),
      title: title.trim(),
      status: 'todo',
      createdAt: new Date().toISOString()
    };
    db.prepare('INSERT INTO tasks VALUES(@id,@title,@status,@createdAt)').run(task);
    return task;
    // npm test -> STILL GREEN after adding persistence ✓
  },
  findById: (id) => db.prepare('SELECT * FROM tasks WHERE id=?').get(id),
  findAll: (s) => s ? db.prepare('SELECT * FROM tasks WHERE status=?').all(s) : db.prepare('SELECT * FROM tasks').all(),
};
module.exports = Task;
```

TDD Pros and Cons

Pros	Cons
Forces thinking about requirements before writing any code	Slower upfront — writing tests first requires discipline
Built-in safety net — refactoring is fearless	Hard to apply to UI, databases, and integration boundaries
Tests serve as living executable documentation	Developers may write tests to pass, not to specify
Lower defect rates in the long run	Requires team-wide discipline and good tooling
Encourages small, focused, independently testable functions	Can produce over-mocked, brittle test suites

TDD vs SDD: In TDD you write a test before code. In SDD you write a full spec before both. SDD subsumes TDD — Phase 4 (Validate) produces tests, but they are derived from a CLAUDE.md spec rather than from the developer's mental model alone.

BDD — Behaviour-Driven Development

Given/When/Then · Gherkin · Cucumber JS example

Definition

Behaviour-Driven Development (BDD) is an extension of TDD that shifts the language of tests from technical assertions to business-readable scenarios. BDD scenarios are written in **Gherkin syntax** — a structured English format using **Given / When / Then** clauses — so that non-technical stakeholders (product managers, QA, business analysts) can read, write, and validate specifications. Tools like **Cucumber** (JavaScript/Java/Ruby), **Behave** (Python), and **SpecFlow** (.NET) execute Gherkin scenarios as automated tests. BDD was pioneered by Dan North in 2006.

Gherkin Keywords Explained

Keyword	Purpose	Example
Feature	Describes a high-level capability	Feature: Task Management API
Scenario	A single test case with context	Scenario: Creating a task successfully
Given	Pre-condition — system state before action	Given the API is running on port 3000
When	The action performed by user or system	When I POST /tasks with {"title":"Buy milk"}
Then	The expected outcome — assertion	Then the response status should be 201
And	Additional Given, When, or Then clause	And the response body should have status "todo"
But	Negative additional clause	But the response should not contain a password field

BDD — Full Example (Tasks API)

tasks.feature — Written in Gherkin (business-readable)

```
# tasks.feature — written by product manager, QA, and developer together
Feature: Task Management API
  As a user of the Tasks API I want to manage my tasks via HTTP endpoints So that I can track my work programmatically
  Scenario: Successfully creating a task
    Given the Tasks API is running
    When I send a POST to /tasks with body {"title": "Buy milk"}
    Then the response status should be 201
    And the response body should contain status "todo"
    And the response body should contain a valid UUID id
  Scenario: Rejecting a task with no title
    Given the Tasks API is running
    When I send a POST to /tasks with an empty body
    Then the response status should be 400
    And the response body error should be "Invalid title"
  Scenario: Fetching a task that does not exist
    Given the Tasks API is running
    When I send a GET to /tasks/nonexistent-uuid
    Then the response status should be 404
  Scenario: Deleting a task
    Given a task exists with title "Buy milk"
    When I send a DELETE to /tasks/{task.id}
    Then the response status should be 204
    And a subsequent GET to /tasks/{task.id} returns 404
```

step_definitions.js — Cucumber maps Gherkin to code

```
// features/step_definitions/tasks.steps.js
const { Given, When, Then } = require('@cucumber/cucumber');
const request = require('supertest');
const app =
```

```
require('.../app'); let response; let taskId; Given('the Tasks API is running', () => { /*
app auto-loaded */ }); When('I send a POST to \/tasks with body {string}', async (body) => {
response = await request(app).post('/tasks').send(JSON.parse(body)); }); When('I send a POST
to \tasks with an empty body', async () => { response = await
request(app).post('/tasks').send({}); }); When('I send a GET to \tasks\/{string}', async (id)
=> { const realId = id === 'nonexistent-uuid' ? id : taskId; response = await
request(app).get(`/tasks/${realId}`); }); When('I send a DELETE to \tasks\/{string}', async
() => { response = await request(app).delete(`/tasks/${taskId}`); }); Then('the response
status should be {int}', (status) => { expect(response.status).toBe(status); }); Then('the
response body should contain status {string}', (status) => {
expect(response.body.status).toBe(status); taskId = response.body.id; // save for subsequent
steps }); Then('the response body error should be {string}', (msg) => {
expect(response.body.error).toBe(msg); });
```

BDD Pros and Cons

Pros	Cons
Business-readable — product managers can write scenarios	Gherkin can become verbose and hard to maintain
Living documentation that stays in sync with tests	Step definitions require boilerplate mapping code
Forces team collaboration before development starts	Setup overhead of Cucumber/Behave/SpecFlow tooling
Scenarios serve as shared acceptance criteria	Teams often skip non-technical stakeholder collaboration
Makes 'what' explicit before 'how' is considered	Can produce fragile end-to-end test suites

BDD vs SDD: BDD uses natural language (Gherkin) to describe expected behaviour. SDD uses structured markdown (CLAUDE.md) to describe system requirements. SDD additionally drives AI code generation — the spec is not just documentation, it is the implementation driver.

Ch 04

DDD — Domain-Driven Design

Ubiquitous language · Bounded contexts · Full JS example

Definition

Domain-Driven Design (DDD) is a software design approach coined by Eric Evans in *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003). In DDD, the code structure mirrors the business domain. DDD establishes a **ubiquitous language** shared by developers and domain experts, and organises the codebase into **bounded contexts** that reflect real-world business boundaries. The domain model — not the database or UI — is the heart of the application.

Core DDD Concepts

- **Ubiquitous Language** — a shared vocabulary agreed upon by developers and business experts. The same terms used in business meetings appear in class names, method names, and variables. If the business says 'Task', the code has a Task class — never a 'Record' or 'Item'.
- **Bounded Context** — a clear boundary within which a domain model is valid. Different contexts may use the same word with different meanings. 'Task' in Projects = a work item; 'Task' in Scheduler = a cron job.
- **Entity** — an object with a unique identity that persists over time. Identity matters more than attributes. A Task entity with the same UUID is the same task even if its title changes.
- **Value Object** — an object defined entirely by its attributes, with no identity. Immutable. Example: TaskStatus ('todo', 'in_progress', 'done') is a value object — two TaskStatus.TODO values are equal.
- **Aggregate** — a cluster of entities and value objects treated as a single unit with one aggregate root. All access to the cluster goes through the root.
- **Repository** — an abstraction over data persistence providing a collection-like interface. The domain layer never talks to the database directly — only through a Repository.
- **Domain Service** — stateless business logic that does not belong to a single entity. Example: TaskPriorityCalculator that ranks tasks across multiple projects.

DDD — Full Code Example (Tasks Domain)

domain/taskStatus.js — Value Object

```
// domain/taskStatus.js — Value Object (immutable, no identity) const TaskStatus =
Object.freeze({ TODO: 'todo', IN_PROGRESS: 'in_progress', DONE: 'done', // Domain rule: can
only transition todo→in_progress→done validTransitions: { 'todo': ['in_progress'],
'in_progress': ['done'], 'done': [] // terminal state } }); module.exports = TaskStatus;
```

domain/task.js — Entity with Ubiquitous Language

```
// domain/task.js — Entity (has identity — UUID) const { v4: uuidv4 } = require('uuid'); const
TaskStatus = require('./taskStatus'); class Task { constructor(id, title, status =
TaskStatus.TODO) { if (!title || title.length > 200) throw new Error('Invalid title'); this.id
= id || uuidv4(); // Entity identity — UUID this.title = title; // Ubiquitous language term
this.status = status; // Value Object this.createdAt = new Date().toISOString(); } // Domain
methods use business language startWork() { const allowed =
TaskStatus.validTransitions[this.status]; if (!allowed.includes(TaskStatus.IN_PROGRESS))
```

```
throw new Error(`Cannot start from ${this.status}`); this.status = TaskStatus.IN_PROGRESS; }
complete() { const allowed = TaskStatus.validTransitions[this.status]; if
(!allowed.includes(TaskStatus.DONE)) throw new Error(`Cannot complete from ${this.status}`);
this.status = TaskStatus.DONE; } isCompleted() { return this.status === TaskStatus.DONE; } }
module.exports = Task;
```

domain/taskRepository.js — Repository Interface

```
// domain/taskRepository.js – Abstract Repository (domain layer contract)
class TaskRepository {
  findById(id) { throw new Error('Not implemented'); }
  save(task) { throw new Error('Not implemented'); }
  findAll(filter) { throw new Error('Not implemented'); }
  delete(id) { throw new Error('Not implemented'); }
  findByStatus(status) { throw new Error('Not implemented'); }
}
module.exports = TaskRepository;
```

infrastructure/sqliteTaskRepository.js — Concrete Implementation

```
// infrastructure/sqliteTaskRepository.js – Implements the domain contract
const TaskRepository = require('../domain/taskRepository');
const Task = require('../domain/task');
const db = require('../db/schema');
class SqliteTaskRepository extends TaskRepository {
  findById(id) {
    const row = db.prepare('SELECT * FROM tasks WHERE id=?').get(id);
    return row ? new Task(row.id, row.title, row.status) : null;
  }
  save(task) {
    db.prepare(
      'INSERT OR REPLACE INTO tasks VALUES(@id,@title,@status,@createdAt)'
    ).run({
      id: task.id,
      title: task.title,
      status: task.status,
      createdAt: task.createdAt
    });
  }
  findAll(filter) {
    const rows = filter?.status ? db.prepare('SELECT * FROM tasks WHERE status=?').all(filter.status) :
    db.prepare('SELECT * FROM tasks').all();
    return rows.map(r => new Task(r.id, r.title, r.status));
  }
  delete(id) {
    db.prepare('DELETE FROM tasks WHERE id=?').run(id);
  }
}
module.exports = SqliteTaskRepository;
```

DDD Bounded Contexts

Bounded Context	What 'Task' means	Key Entities	Repository
Task Management	A user's to-do item	Task, TaskList, User	TaskRepository
Notifications	A scheduled reminder	Notification, Schedule	NotificationRepo
Analytics	A productivity data point	DataPoint, Report	AnalyticsRepo
Integrations	An external sync record	ExternalTask, SyncLog	SyncRepository

DDD Pros and Cons

Pros	Cons
Code mirrors the business — easier for domain experts to review	High learning curve — many patterns to master
Bounded contexts prevent monolithic 'big ball of mud' designs	Overkill for simple CRUD applications
Ubiquitous language reduces costly developer-stakeholder miscommunication	Requires deep ongoing collaboration with domain experts
Domain model is independently testable from infrastructure	Repository pattern adds abstraction layers
Scales well as domain complexity and team size grow	Significant upfront design investment required

DDD + SDD: DDD shapes architecture around the business domain. SDD uses a written spec to drive AI-generated code. They complement each other: write a DDD-informed CLAUDE.md with domain-correct entity names and bounded contexts, then let SDD implement it.

Ch 05

SDD — Spec-Driven Development

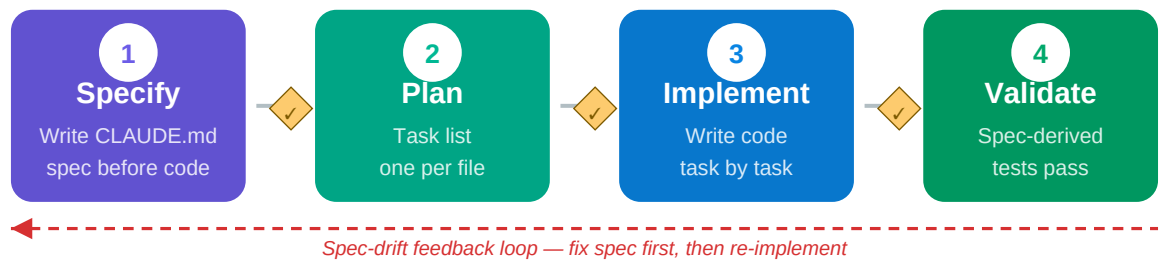
Definition · 5 principles · Pipeline · vs TDD/BDD/DDD

Definition

Spec-Driven Development (SDD) is a methodology in which a written specification is authored and agreed upon *before* any code is generated. The spec — stored in **CLAUDE.md** — defines what the system must do: endpoints, schemas, error codes, constraints. An AI agent (Claude Code) reads this spec and uses it as the authoritative source of truth to drive all four phases: **Specify** → **Plan** → **Implement** → **Validate**.

The SDD 4-Phase Pipeline

Figure 5.1 — Four SDD phases with review gates (◆) and the spec-drift feedback loop.

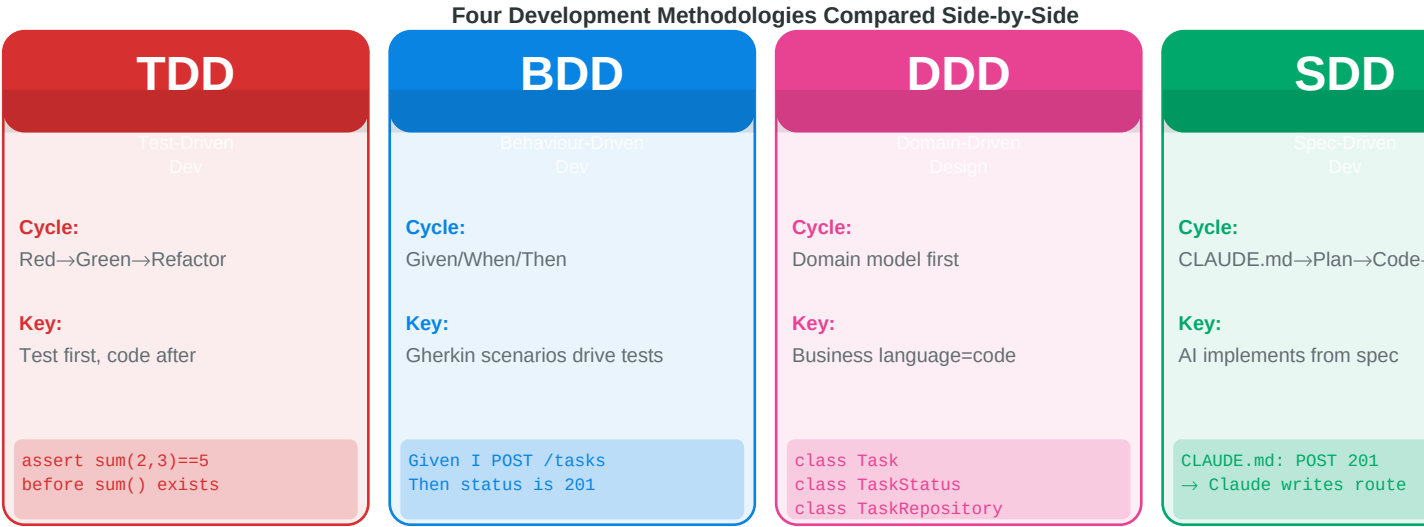


The 5 Principles of SDD

- **Spec first** — CLAUDE.md is written and approved before any tool call runs. No code is generated until the spec is complete and reviewed.
- **Spec as truth** — if code and spec disagree, the spec wins. Fix the code to match the spec, never the other way around.
- **Structured gates** — three deliberate human checkpoints replace continuous micro-review. Decisions happen at the right abstraction level.
- **Traceable tests** — every test maps to a named spec constraint, not to code that already exists.
- **Spec evolves** — future changes update CLAUDE.md before touching the code. The spec is the primary artifact.

All Four Methodologies Compared

Figure 5.2 — TDD, BDD, DDD, and SDD — starting artifact, cycle, key idea, and code example.



Dimension	TDD	BDD	DDD	SDD
Starts with	Failing test	Gherkin scenario	Domain model	CLAUDE.md spec
Written in	Code assertions	English (Gherkin)	UML / classes	Markdown
Non-tech stakeholders?	No	Yes — Gherkin	Partially	No (but readable)
AI drives implementation?	No	No	No	Yes — primary driver
Context persists?	In test suite	In .feature files	In domain model	CLAUDE.md auto-load
Review cadence	Per test cycle	Per scenario	Per model iteration	3 structured gates
Tests derived from	Developer's mental model	Gherkin scenarios	Domain behaviour	CLAUDE.md spec lines
Best for	Unit functions	Acceptance testing	Complex domains	AI-built full features

Ch 06

Core Terminology A–Z

30+ terms every SDD practitioner must know

Agent

CORE

An autonomous AI process with full tool access that can plan, execute tool calls, write files, run commands, and iterate on its own output to complete a multi-step goal. In Claude Code, the entire CLI session is an agent.

Example: Running 'claude' starts an agent that reads `CLAUDE.md`, generates the plan, implements all 9 tasks, and runs tests.

Agentic Loop

CORE

The repeating plan→execute→observe→revise cycle inside an agent. Each iteration: plan the next step → call a tool → read the result → adjust the plan → repeat until goal is achieved or a human intervenes.

Example: Claude reads `CLAUDE.md` → writes `schema.js` → sees a syntax error → self-corrects → continues — one agentic loop.

Bash Tool

TOOL

Built-in Claude Code capability that executes shell commands on the developer's machine — npm install, git commands, test runners, linters, custom scripts. The mechanism through which Claude interacts with the project environment.

Example: Claude runs 'npm test' after T9 and reads the output to verify all 9 tests pass before reporting success.

CLAUDE.md

SPEC

The markdown file that Claude Code auto-loads at every session start. In SDD it is the persistent specification — the single source of truth. Answers every question Claude might otherwise guess at: endpoints, schemas, error codes, constraints, tech stack.

Example: Spec line 'DELETE /tasks/:id returns 204' drives both the route code and the test case for that endpoint.

Context Window

ARCH

The maximum number of tokens Claude can read and reason about in one session. `CLAUDE.md`, tool outputs, and conversation history all consume context. Long sessions may drop early content if the window is exceeded. Plan accordingly.

Example: Claude Sonnet has a 200,000-token context window — roughly 150,000 words or 500 pages of text.

Edit Tool

TOOL

Built-in capability to read and write files on disk. The mechanism through which Claude produces every source file during the Implement phase. Can create new files, modify existing ones, or delete files within allowed permissions.

Example: Claude uses the edit tool to write `routes/tasks.js` for tasks T4 and T5 during the Implement phase.

Gate / Review Gate

PROCESS

A deliberate human checkpoint between SDD phases. Claude Code stops and waits for explicit human approval before continuing. Three gates: after Specify, after Plan, and after Implement.

Example: At Gate 2, you read the 9-task plan and ask 'Should T6 be middleware or inline in the route?' before approving.

Guardrails

SAFETY

Configurable rules constraining what Claude Code is allowed to do — block deleting files, require approval for git push, prevent network requests outside allowed domains. Protect against unintended side effects in agentic sessions.

Example: Guardrail: require human approval before any command matching 'rm -rf' or 'git push --force'.

Hook

AUTOMATION

A shell command configured to fire automatically at a lifecycle event: PreToolUse (before every tool call), PostToolUse (after every tool call), Stop (when Claude finishes), Notification (when user input needed). The primary automated quality gate mechanism in SDD.

Example: PostToolUse hook: after every file write → run 'eslint {file} --fix'. Claude sees lint errors and self-corrects before continuing.

Instruction Hierarchy

ARCH

Priority order for conflicting instructions: System prompt (operator) > CLAUDE.md (project spec) > User message > Tool output. Higher levels always override lower. CLAUDE.md overrides anything you type mid-session.

Example: CLAUDE.md says 'no auth in v1'. You type 'add JWT now'. Claude flags the conflict with the spec and stops.

MCP — Model Context Protocol

PROTOCOL

Open standard (JSON-RPC 2.0) connecting Claude Code to external tools and data sources — databases, APIs, browsers, calendars, file systems. Any MCP-compatible server can be used by any MCP-compatible client without custom integration.

Example: PostgreSQL MCP server lets Claude query your database directly during the Validate phase without manual setup.

Memory — In-Context

PERSIST

Everything in the current session — messages, file contents read, tool outputs. Exists only in the context window. Lost when the session ends. This is why SDD relies on CLAUDE.md rather than conversation history for persistence.

Example: Claude reads tasks.js mid-session and 'remembers' its content for the rest of that session only.

Memory — Project

PERSIST

Facts stored in CLAUDE.md and sub-spec files. Persists in the git repository. Auto-loaded by Claude Code at every session start. The primary SDD persistence mechanism — survives session restarts, new terminals, and team handoffs.

Example: Session starts tomorrow. Claude reads CLAUDE.md and instantly knows the full schema without re-explanation.

Memory — User

PERSIST

Facts stored in the claude.ai account about the developer — preferred stack, coding style, communication preferences. Persists across all projects and sessions automatically. Applied silently to every new conversation.

Example: Claude remembers you prefer concise code comments and applies that style automatically in every session.

Multi-agent

ARCH

A system where multiple Claude Code agents run in parallel on the same project. An orchestrator reads the spec and assigns task groups to sub-agents. All agents read the same CLAUDE.md, ensuring coherence despite parallel execution.

Example: Orchestrator assigns T2-T5 to Agent A and T6-T8 to Agent B — both implement their task groups simultaneously.

Orchestrator

ROLE

The top-level agent in a multi-agent system that reads CLAUDE.md and the task list, delegates task groups to sub-agents, monitors their outputs, and assembles the final result. Does not write code itself — coordinates those that do.

Example: 'Task T3 complete by Agent A. Assign T4 to Agent B. Merge both outputs when complete, then run tests.'

Permissions

SAFETY

The set of actions Claude Code is allowed to perform in a session — read files, write to specific directories, run bash commands, use specific MCP servers. Configured in .claude/settings.json and enforced by the permissions layer.

Example: Permission set: read all files, write only to src/ and tests/, no network calls, no git push.

Skill

KNOWLEDGE

A SKILL.md file encoding expert instructions for a specific task type. Claude Code reads the relevant skill before any task of that type. Mandatory — never skipped. Skills encode library constraints, environment quirks, and output quality rules not in Claude's training.

Example: User asks for a Word doc → Claude reads /mnt/skills/public/docx/SKILL.md before writing a single line of Python.

Spec

CORE

Short for specification. The written document (CLAUDE.md in SDD) that precisely defines what a system must do — endpoints, schemas, error codes, constraints. Everything else — code, tests, docs — must agree with the spec.

Example: Spec line: 'POST /tasks returns 201'. That single line drives the route code AND the test case.

Spec Drift

ANTI-PATTERN

The gradual divergence between the written spec and the actual implemented code. Happens when code is patched or extended without updating CLAUDE.md. Makes future AI-assisted sessions unreliable because Claude reads a stale spec.

Example: Dev patches 200→300 char limit in code but forgets CLAUDE.md. Next session Claude re-enforces 200 chars — a production bug.

Sub-agent

ARCH

A Claude Code agent spawned by an orchestrator to handle a specific task group in parallel with other sub-agents. Has access to the same tools (edit, bash, MCP) and reads the same CLAUDE.md, ensuring spec-aligned output.

Example: Sub-agent A implements model layer (T2,T3) while Sub-agent B implements routes (T4,T5) simultaneously.

System Prompt

ARCH

Hidden operator instructions injected before the session starts. Configures role, allowed tools, persona, and guardrails. In Claude Code, set via --system-prompt flag or operator settings. Highest priority in the instruction hierarchy.

Example: claude --system-prompt='You are a backend engineer. Only edit files in src/. Never run git push.'

Task List

ARTIFACT

Numbered list of implementation tasks produced in Phase 2 (Plan). One task per file or logical unit. Small enough to implement, review, and test independently. The bridge between the spec and the code.

Example: T1 package.json · T2 db/schema.js · T3 models/task.js · T4-T5 routes · T6-T7 middleware · T8 app.js · T9 tests

Tool Call

ARCH

Claude's structured request to use a capability — edit (write a file), bash (run a command), web_search (search the internet), or an MCP tool (call an external service). The mechanism through which Claude takes action in the world.

Example: {tool:'edit', file:'routes/tasks.js', content:'...'} → edit tool executes → Claude reads result and continues.

Traceability

QUALITY

The property that every requirement in CLAUDE.md can be followed through the task list, into the code, through the test, and all the way to a passing test result. Broken traceability indicates a spec gap or implementation gap.

Example: Spec '400 if title missing' → T6 (validate.js) → route code → test 'POST 400 no title' → PASS.

Validate

PHASE

Phase 4 of SDD. Claude writes integration tests derived from spec constraints and runs them. Every test maps to a named spec rule. All must pass. Failures require returning to Specify to correct the spec, then re-implementing the affected tasks.

Example: Test: 'POST /tasks → 400 if title over 200 chars' — maps directly to the CLAUDE.md error rule.

Web Search Tool

TOOL

Built-in Claude Code tool that performs live internet searches and returns results. Used when the spec requires knowledge of external APIs, library documentation, or best practices beyond Claude's training cutoff date.

Example: Claude searches 'better-sqlite3 prepared statements api' before writing db/schema.js to verify the correct API.

Ch 07

Claude Code Architecture

CLI · Agent loop · All tools · Session lifecycle · Context tips

Claude Code is a command-line agentic coding environment. Every component plays a distinct role in the SDD workflow. Understanding the full architecture helps you design better CLAUDE.md specs, configure hooks and guardrails correctly, and debug unexpected agent behaviour.

Component Overview

Component	What it is	SDD role
CLI (claude)	Command-line session launcher	Entry point; reads CLAUDE.md automatically on start
Agent loop	Plan→execute→observe→revise cycle	Runs all 9 tasks from the plan sequentially or in parallel
Edit tool	Read/write files on disk	Produces every source file and updates to CLAUDE.md
Bash tool	Run any shell command	Installs deps, runs tests, invokes linters and build scripts
Web search tool	Live internet search	Looks up library docs, error messages, current API specs
MCP client	Connects to MCP servers	Integrates databases, APIs, browsers, external services
CLAUDE.md loader	Auto-reads spec at session start	Injects full spec into every session's context window
Hooks engine	Fires shell commands on events	Automated quality gates after file writes or on Stop
Permissions layer	Enforces allowed/blocked actions	Prevents unsafe operations on production systems
Sub-agent spawner	Creates parallel worker agents	Speeds up large Implement phases significantly
Computer use	Controls GUI applications	UI testing, browser automation in Validate phase

Session Lifecycle — Step by Step

- 1. Run 'claude' in the project root directory.
- 2. Claude Code scans for CLAUDE.md and loads it into the context window automatically.
- 3. You issue Phase 2: 'Generate a numbered implementation plan. One task per file. No code yet.'
- 4. Claude enters the agentic loop: plan → tool calls → observe results → revise → report.
- 5. Hooks fire at configured events (PostToolUse after every file write; Stop when Claude finishes).
- 6. You review at Gate 2 and approve or correct the task list before implementation begins.
- 7. Claude implements tasks in batches, stopping at Gate 3 for your spec-to-code verification.
- 8. Claude writes T9 tests and runs them with the bash tool. All 9 must pass.
- 9. Session ends. CLAUDE.md persists in git. All other context is discarded.

Context Window Management

- Keep CLAUDE.md concise — use it as an index referencing /specs/ sub-files for large projects.
- Use /clear between major phases to reset conversation while CLAUDE.md persists automatically.
- Implement in batches of 2-3 tasks — do not ask Claude to read 15 files in one chain.

- Large binary files should never be loaded into context — reference them via MCP filesystem tools.

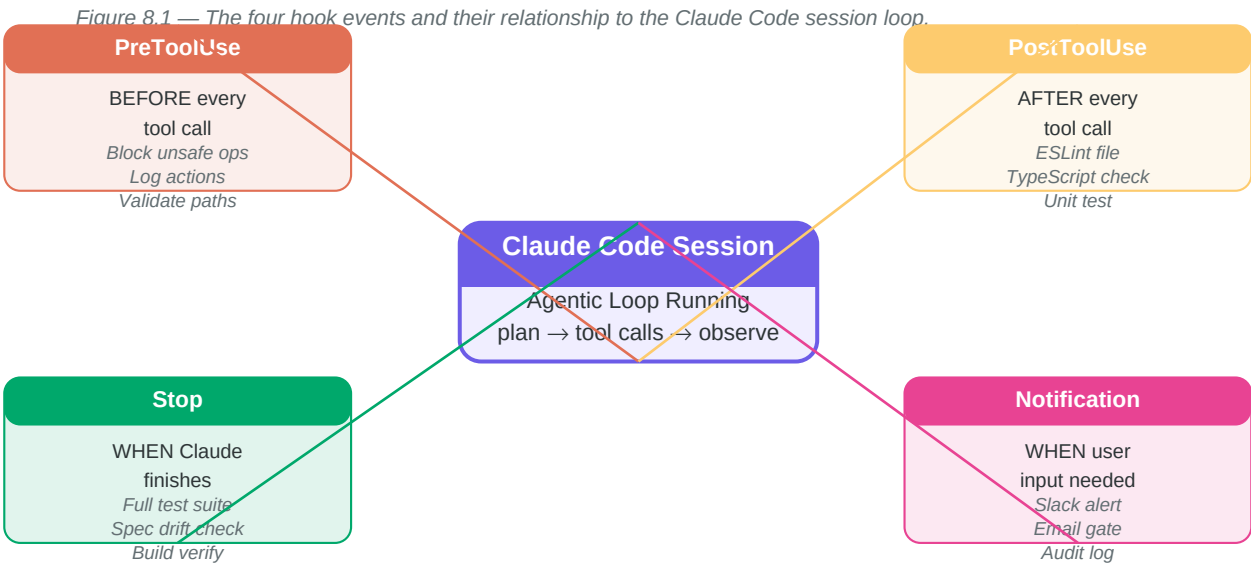
Ch 08

Hooks — Complete Reference

4 events · Lifecycle diagram · Config · Patterns · Safety

Hooks are shell commands that Claude Code executes automatically at specific lifecycle events. They are the primary mechanism for enforcing automated quality gates — linting, type-checking, test running, and team notifications — without any manual intervention between tasks.

Hook Lifecycle Event Diagram



The Four Hook Events

Event	Fires when	Primary SDD use cases	Example command
PreToolUse	Before every tool call	Validate path · Log action · Check permissions	echo '{file}' >> audit.log
PostToolUse	After every tool call	ESLint · TypeScript check · Unit test modified files	eslint {file} --fix --quiet
Stop	Claude finishes a task	Full test suite · Spec drift check · Build verify	npm test -- --silent
Notification	User input needed	Slack alert · Email gate · Review gate notification	curl -X POST \$HOOK -d 'gate'

Hook Configuration — .claude/settings.json

```
{ "hooks": { "PreToolUse": [ { "matcher": "bash", "command": "echo 'BASH: {command}' >> .claude/audit.log" } ], "PostToolUse": [ { "matcher": "edit", "command": "eslint {file} --fix --quiet" }, { "matcher": "edit", "command": "npx tsc --noEmit 2>&1 | head -20" } ], "Stop": [ { "command": "npm test -- --silent" }, { "command": "node scripts/check-spec-drift.js" } ], "Notification": [ { "command": "curl -s -X POST $SLACK_WEBHOOK -d 'Review gate reached'" } ] }
```

Hook Design Patterns for SDD

- **Lint-on-write** — ESLint or Prettier fires after every file write. Style violations caught and auto-fixed before Claude continues to the next task.

- **Type-check-on-write** — `tsc --noEmit` fires after every TypeScript file write. Type errors surfaced immediately.
- **Test-on-stop** — full test suite runs automatically when Claude finishes the Implement phase (Stop event).
- **Spec-check script** — custom script verifying every endpoint in `CLAUDE.md` has a matching Express route in the codebase.
- **Audit log** — `PreToolUse` appends every bash command to a log file for full session observability and audit trail.
- **Gate notification** — Notification event posts a Slack message when Claude is waiting at a review gate for human approval.

Safety: hooks run with your user permissions. A hook that deletes files or runs 'git push' can cause irreversible damage. Test all hooks on a non-production branch first. Use guardrails alongside hooks.

Ch 09

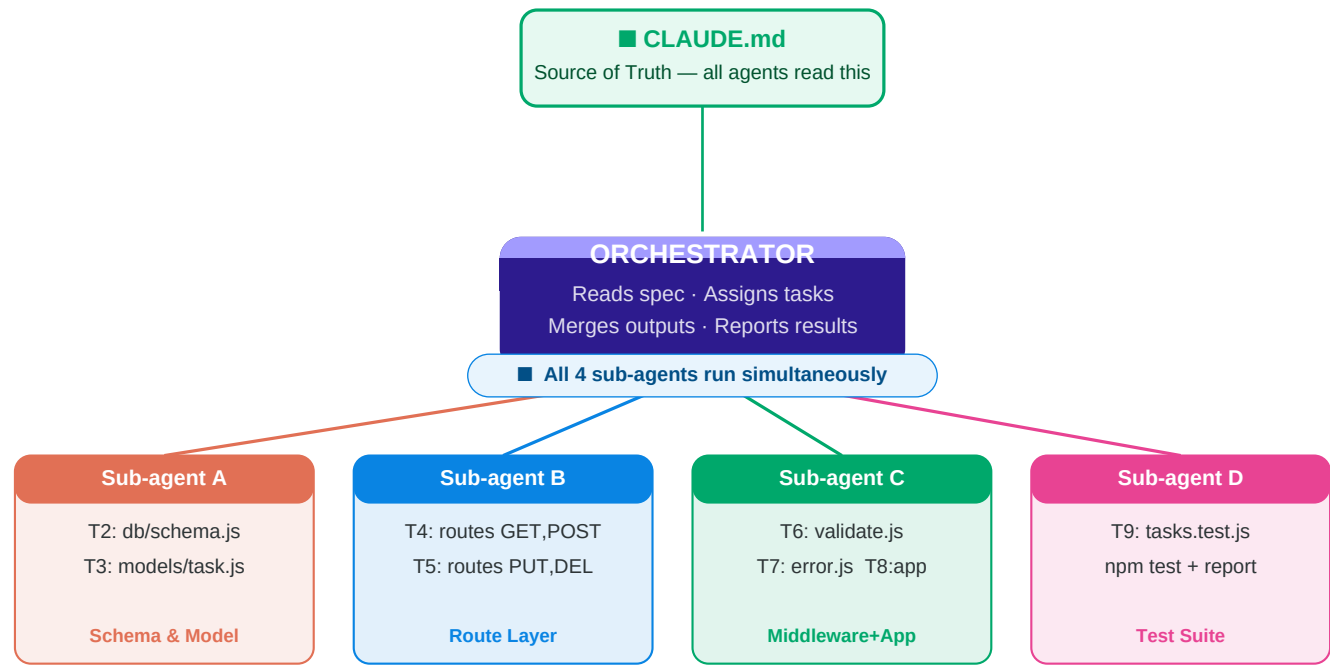
Agents, Sub-agents & Centralized Orchestration

Full multi-agent SDD architecture & coherence rules

An agent is an autonomous AI process with full tool access that plans and executes a multi-step task without per-step human approval. Sub-agents are spawned by an orchestrator to handle task groups in parallel. All agents read the same CLAUDE.md, ensuring coherence.

Centralized Orchestration Diagram

Figure 9.1 — Orchestrator reads CLAUDE.md and assigns task groups to 4 parallel sub-agents.



Agent Roles in SDD

Role	Responsibilities	Example in Tasks API
Orchestrator	Read spec, plan tasks, assign to sub-agents, merge, report	Reads CLAUDE.md, assigns T2-T5 to A, T6-T8 to B, T9 to D
Sub-agent A	Implements assigned task group	db/schema.js + models/task.js (schema + model layer)
Sub-agent B	Implements assigned task group	routes/tasks.js — all 5 endpoints (T4 + T5)
Sub-agent C	Implements assigned task group	validate.js + error.js + app.js
Sub-agent D	Writes and runs tests	tasks.test.js — 9 spec-derived tests + npm test

Orchestrator Prompt Example

```
claude "You are the orchestrator. Read CLAUDE.md. Spawn sub-agent-a: implement T2 (db/schema.js) and T3 (models/task.js). Spawn sub-agent-b: implement T4 and T5 (routes/tasks.js all 5 endpoints). Spawn sub-agent-c: implement T6 (validate.js), T7 (error.js), T8 (app.js). Wait for all three agents to complete. Spawn sub-agent-d: implement T9 and run npm test. Report which CLAUDE.md constraint each passing test validates."
```

What Keeps Multi-agent SDD Coherent?

- **CLAUDE.md** — every agent reads the same spec, so all produce code meeting identical constraints (status codes, schema field names, error shapes).
- **Task boundaries** — non-overlapping file assignments prevent merge conflicts between parallel agents.
- **Interface contracts** — the spec defines the model API shape (Task.create, Task.findById) so route agents call model functions correctly.
- **Test validation** — Sub-agent D catches any inconsistency between the other agents' outputs before the session ends.
- **Hooks** — ESLint on every write enforces consistent code style across all parallel agents automatically.

Single-agent vs Multi-agent SDD

Aspect	Single-agent	Multi-agent
Execution	Sequential (T1→T2→...→T9)	Parallel (A+B+C simultaneously)
Speed	Standard for small projects	Up to 4x faster for 9+ task plans
Setup complexity	Low — one 'claude' command	Higher — orchestrator prompt needed
Best for	1-9 task projects	10+ task projects, large APIs

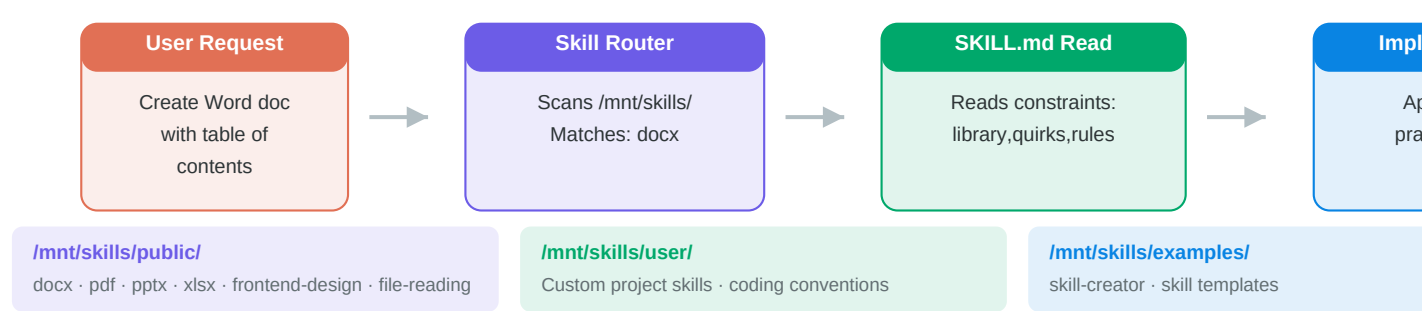
Skills — Types, Anatomy & Creation

Routing diagram · SKILL.md anatomy · All public skills

A **skill** is a reusable SKILL.md file encoding expert instructions for a specific task type. Claude Code reads the relevant skill before any task of that type. Skills encode hard-won knowledge about library constraints, environment quirks, and output quality requirements that are not reliably in Claude's training data. Reading the skill is **mandatory**.

Skill Routing Flow Diagram

Figure 10.1 — How a user request routes through the skill system to implementation.



SKILL.md Anatomy

```
--- name: tasks-api-conventions description: Use for any work on the Tasks REST API project.
triggers: ['tasks api', 'task model', 'task route', 'task endpoint'] --- # Tasks API – Coding
Conventions ## Required libraries (do not substitute) - Express 4.x for routing (not Fastify,
not Koa) - better-sqlite3 for SQLite (NOT the 'sqlite3' package – different API) - uuid v4:
const { v4: uuidv4 } = require('uuid'); ## Database rules – CRITICAL Always prepared
statements: db.prepare('SELECT...').all() NEVER string-interpolate SQL – SQL injection risk.
## Error response shape Always: { error: 'message string' } – never plain strings, never
nested. ## Status codes (must match CLAUDE.md exactly) POST creates:201 GET/PUT success:200
DELETE:204 Not found:404 Bad input:400
```

All Built-in Public Skills

Skill	Trigger	What it covers
docx	Create Word document	python-docx, TOC, page numbers, heading styles, column layouts
pdf	Create or fill PDF	reportlab vs pypdf, form filling, page manipulation, OCR
pptx	Create presentation slides	python-pptx, slide layouts, master slides, charts, animations
xlsx	Create spreadsheet	openpyxl, formulas, charts, conditional formatting, data validation
frontend-design	React/HTML/Vue components	Design tokens, Tailwind classes, responsive patterns, accessibility
file-reading	Read any uploaded file	Which tool per file type, encoding handling, large file strategy
pdf-reading	Extract PDF content	Page rasterization, text extraction, form fields, table detection

Skill	Trigger	What it covers
product-self-knowledge	Claude API & claude.ai	Models, pricing, API parameters, SDK usage, rate limits

Mandatory rule: before writing any file or running any command, Claude Code reads the relevant SKILL.md. Skipping a skill read is the single most common cause of poor output quality.

Ch 11

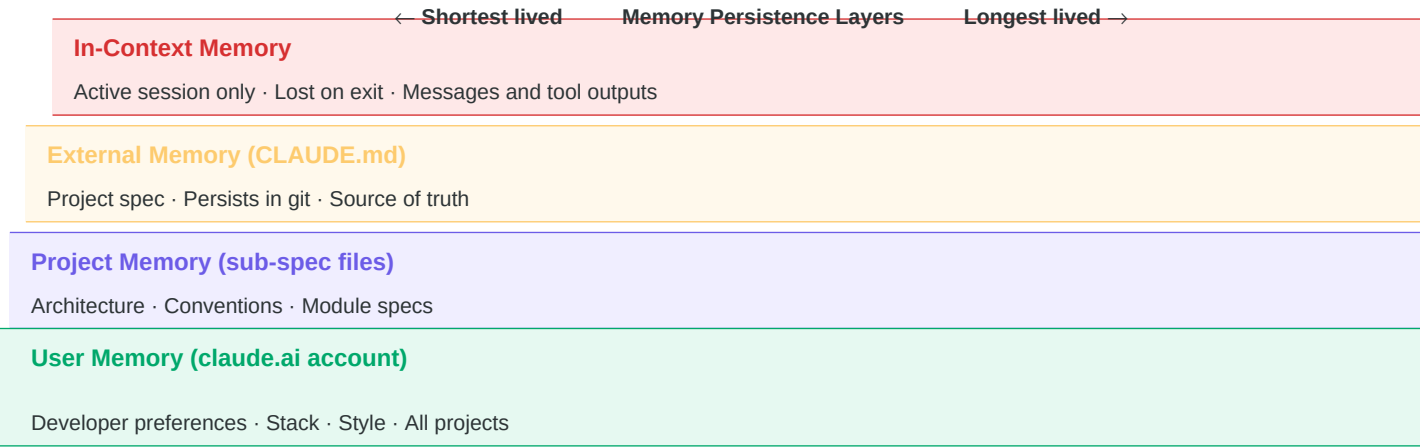
Memory — All 4 Layers

Pyramid diagram · What to store · Multi-file CLAUDE.md pattern

Claude Code has four distinct memory layers with different scope, persistence, and SDD roles. Understanding all four is essential for designing workflows that survive session restarts, team handoffs, and long-lived projects.

Memory Persistence Pyramid

Figure 11.1 — Four memory layers from ephemeral (top) to permanent (bottom).



Layer	Scope	Storage	Survives session?	SDD role
In-context	Current session only	Context window (tokens)	No — lost on exit	Active messages + tool outputs
External (CLAUDE.md)	One project	File in git repository	Yes — auto-loaded	THE spec — source of truth
Project (sub-specs)	One project	Files in git repository	Yes — versioned	Architecture, conventions, modules
User (claude.ai)	All projects/sessions	claude.ai account	Yes — permanent	Developer preferences, stack, style

What to Store in CLAUDE.md (Project Memory)

- Project goal in one sentence with the target audience.
- Tech stack with exact library versions (Node.js 20, Express 4.18.2, better-sqlite3 9.4.3).
- All endpoints with HTTP method, path, request body, response body, and all status codes.
- All data schema fields with name, type, constraints (max length, required), and defaults.
- All error rules with HTTP code and exact trigger condition in plain English.
- Coding conventions — naming patterns, SQL style, error response shape, file structure.
- Architecture decisions with rationale — 'SQLite not Postgres: simpler for v1, no connection pool'.
- Known environment gotchas — library quirks, OS-specific constraints, CI environment limits.

Multi-file CLAUDE.md Pattern for Large Projects

```
# CLAUDE.md (root — index only, keep under 50 lines) # Tasks API — Master Spec Index ## Stack:
Node.js 20 | Express 4 | better-sqlite3 | uuid ## Sub-specs See /specs/api.md — all endpoint
```

```
definitions with examples See /specs/schema.md – database schema, field rules & examples See  
/specs/errors.md – all HTTP error codes & trigger conditions See /specs/auth.md –  
authentication & authorization rules (v2) See /specs/conventions.md – coding style, naming,  
SQL rules
```

Ch 12

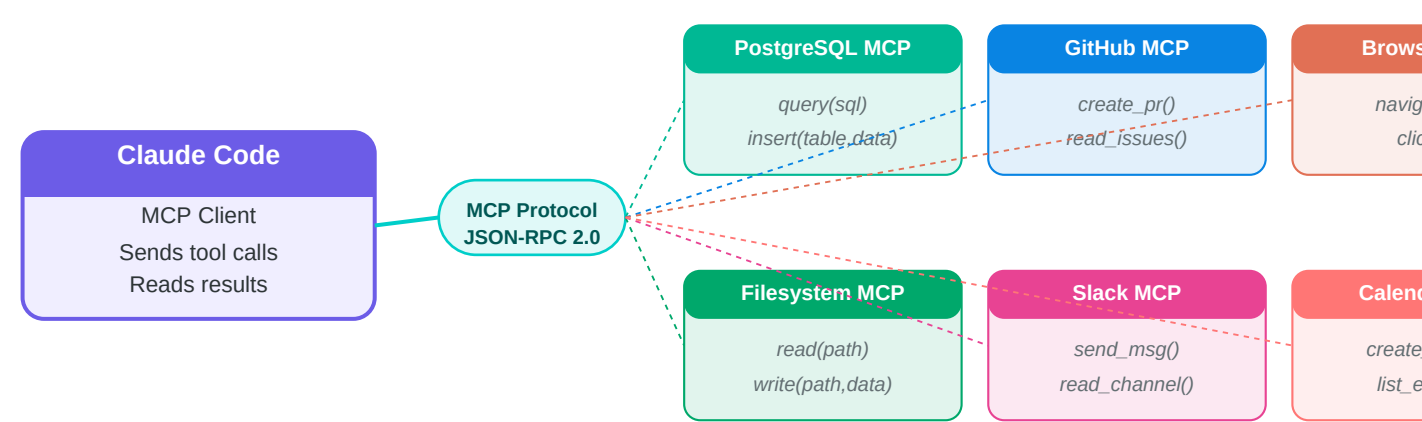
MCP — Model Context Protocol

Architecture diagram · 6 servers · Config · SDD integration

MCP (Model Context Protocol) is an open standard by Anthropic that defines how AI agents connect to external tools and data sources through a standardised JSON-RPC 2.0 interface. Any MCP-compatible server can be used by any MCP-compatible client without custom integration code. MCP is to AI tools what USB is to hardware — one standard for everything.

MCP Architecture Diagram

Figure 12.1 — Claude Code as MCP client connected to 6 MCP servers via the protocol bus.



Component	Role	SDD example
MCP server	Exposes typed tools via JSON-RPC 2.0	PostgreSQL: query(sql), insert(table,data)
MCP client	Claude Code — calls tools on servers	Claude queries DB during Validate phase
Tool	Named, typed function on a server	db_query(sql: string) → Row[] error
Resource	Read-only data source on a server	/schema — DB schema as structured resource
Transport	Client-server communication	stdio (local) or HTTP (remote hosted)

Common MCP Servers for SDD

- **Database MCP** — query PostgreSQL, MySQL, or SQLite. Validate real data shapes after implementation without manual DB access.
- **GitHub MCP** — create issues for spec gaps, open PRs after Validate passes, read issue comments as requirements.
- **Slack MCP** — post review gate notifications to the engineering channel when human approval is needed.
- **Browser MCP** — automate UI testing in the Validate phase without manual browser operation or Selenium setup.
- **Filesystem MCP** — read project files outside the Claude Code working directory, including legacy codebases.

- **Calendar MCP** — schedule review gate meetings when multi-person approval is required for major spec changes.

MCP Configuration

```
// .claude/settings.json { "mcp": { "servers": { "postgres": { "command": "npx  
@modelcontextprotocol/server-postgres", "args": ["postgresql://localhost:5432/tasks_db"] },  
"github": { "command": "npx @modelcontextprotocol/server-github", "env": {  
"GITHUB_PERSONAL_ACCESS_TOKEN": "$GH_TOKEN" } }, "slack": { "command": "npx  
@modelcontextprotocol/server-slack", "env": { "SLACK_BOT_TOKEN": "$SLACK_TOKEN" } } } } }
```

Ch 13

The 4 SDD Phases — Deep Dive

Per-phase guide · CLAUDE.md template · Gate checklists

Each SDD phase produces a concrete artifact that a human reviews and approves before the next phase begins. Decisions happen at the right level of abstraction — requirements reviewed as markdown, plan reviewed as a numbered list, code reviewed file by file.

■ Phase 1 — Specify

What you produce: CLAUDE.md

- Write CLAUDE.md using the minimum viable template. Define every endpoint with method, path, request body, response body, and all status codes.
- Define every schema field with name, type, constraints (max length, required), default value, and an example.
- Write every error rule: trigger condition → HTTP code → response body shape.
- State the tech stack with exact library versions. Leave nothing for Claude to assume.

CLAUDE.md Minimum Viable Template

```
# — specification ## Goal ## Stack ## Endpoints ## Data Schema ## Error Rules ## Constraints
## Coding Conventions
```

Gate 1: Is every endpoint, schema field, and error code documented explicitly? Would a new engineer implement this correctly with only this file?

■ Phase 2 — Plan

What you produce: numbered task list

- Prompt: 'Read CLAUDE.md and produce a numbered implementation plan. One task per file or logical unit. No code yet.'
- Read every task. Trace it to at least one spec line. Add missing tasks, remove duplicates, reorder for correct dependency order.

Gate 2: Does every CLAUDE.md spec line map to at least one task? Is the file creation order correct (db before model before routes)?

■ Phase 3 — Implement

What you produce: all source files

- Ask Claude to implement tasks in batches: 'Implement T1 and T2. Stop for review after each file.'
- At each stop, trace the generated code to the spec constraint. Verify hooks (lint, type check) pass automatically.

Gate 3: Can I trace every CLAUDE.md constraint to a specific line of implementation code?

■ Phase 4 — Validate

What you produce: test suite + all tests passing

- Ask Claude: 'Implement T9 (integration tests). One test per CLAUDE.md constraint. Run npm test. For each failure, cite the spec line.'
- If a test fails, check the spec first — was the constraint incomplete? Fix CLAUDE.md → re-implement → re-test.

After Validate: CLAUDE.md is now both spec and living documentation. Future features follow the same four-phase process — extend the spec before touching the code.

Phase	Input	Claude's action	Output	Gate question
1 Specify	Your requirements	Nothing — you write CLAUDE.md	CLAUDE.md	Every constraint documented?
2 Plan	CLAUDE.md	Read spec → produce task list	Numbered task list	Every spec line has a task?
3 Implement	Task list + CLAUDE.md	Write files task by task	Source files	Code traces to spec?
4 Validate	Source files + CLAUDE.md	Write & run spec-derived tests	Test results + pass/fail	All constraints pass?

Ch 14

Full Tasks API Example

Complete CLAUDE.md · All 8 source files · 9 tests · Results

A complete Tasks REST API built end-to-end using SDD with Claude Code. Node.js 20, Express 4, better-sqlite3, Jest + Supertest. Every file traces to a CLAUDE.md spec line.

■ CLAUDE.md — The Complete Spec

```
# Tasks API — specification ## Goal REST API for user task management. Persists to SQLite.
JSON responses. No auth (v1). ## Stack Node.js 20 | Express 4.18 | better-sqlite3 9.4 | uuid
9.0 | jest 29 ## Endpoints GET /tasks # list; optional ?status=todo|in_progress|done; returns
200 POST /tasks # create; body:{title:string}; returns 201 GET /tasks/:id # fetch one; returns
200; 404 if not found PUT /tasks/:id # update title/status; returns 200; 404 if not found
DELETE /tasks/:id # delete; returns 204; 404 if not found ## Task Schema id TEXT UUID v4,
auto-generated, never user-supplied title TEXT required, max 200 characters status TEXT enum:
todo|in_progress|done default:todo createdAt TEXT ISO 8601 timestamp, set on create, immutable
## Error Rules 400 title missing OR title.length>200 → {error:'Invalid title'} 404 :id not in
database → {error:'Not found'} 500 unhandled server error → {error:'Internal server error'}
```

db/schema.js — T2

```
const Database=require('better-sqlite3'); const db=new Database('tasks.db'); db.exec(`CREATE
TABLE IF NOT EXISTS tasks( id TEXT PRIMARY KEY,title TEXT NOT NULL, status TEXT DEFAULT
'todo',createdAt TEXT NOT NULL )`); module.exports=db;
```

models/task.js — T3

```
const{v4:uuidv4}=require('uuid'); const db=require('../db/schema'); const Task={
findAll:(s)=>s?db.prepare('SELECT*FROM tasks WHERE status=?').all(s):db.prepare('SELECT*FROM
tasks').all(), findById:(id)=>db.prepare('SELECT*FROM tasks WHERE id=?').get(id),
create:(title)=>{ const t={id:uuidv4(),title,status:'todo',createdAt:new
Date().toISOString()}; db.prepare('INSERT INTO tasks
VALUES(@id,@title,@status,@createdAt)').run(t); return t; }, update:(id,f)=>{ const
s=Object.keys(f).map(k=>`${k}=@${k}`).join(','); db.prepare(`UPDATE tasks SET ${s} WHERE
id=@id`).run({...f,id}); return Task.findById(id); }, remove:(id)=>db.prepare('DELETE FROM
tasks WHERE id=?').run(id) }; module.exports=Task;
```

routes/tasks.js — T4 + T5 (all 5 endpoints)

```
const router=require('express').Router(); const Task=require('../models/task');
router.get('/',(q,s)=>s.json(Task.findAll(q.query.status))); router.post('/',(q,s)=>{
const{title}=q.body; if(!title||title.length>200) // spec: 400 error rule return
s.status(400).json({error:'Invalid title'}); s.status(201).json(Task.create(title)); // spec:
201 on POST }); router.get('/:id',(q,s)=>{ const t=Task.findById(q.params.id); if(!t)return
s.status(404).json({error:'Not found'}); // spec:404 s.json(t); }); router.put('/:id',(q,s)=>{
if(!Task.findById(q.params.id))return s.status(404).json({error:'Not found'});
s.json(Task.update(q.params.id,q.body)); }); router.delete('/:id',(q,s)=>{
if(!Task.findById(q.params.id))return s.status(404).json({error:'Not found'});
Task.remove(q.params.id); s.status(204).send(); // spec: 204 on DELETE });
module.exports=router;
```

middleware/validate.js — T6 & middleware/error.js — T7


```
// middleware/validate.js - T6 function validateTask(req,res,next){ const{title}=req.body;
if(!title||title.length>200) return res.status(400).json({error:'Invalid title'}); next(); }
module.exports=validateTask; // middleware/error.js - T7 function
errorHandler(err,req,res,next){ console.error(err.stack);
res.status(500).json({error:'Internal server error'}); } module.exports=errorHandler;
```

app.js — T8

```
const express=require('express'); const taskRoutes=require('./routes/tasks'); const
errorHandler=require('./middleware/error'); const app=express(); app.use(express.json());
app.use('/tasks',taskRoutes); app.use(errorHandler);
if(require.main===module)app.listen(3000,()=>console.log('API on :3000'));
module.exports=app;
```

tests/tasks.test.js — T9 (9 spec-derived tests)

```
const req=require('supertest'),app=require('../app'); describe('Tasks API - spec
validation',()=>{ let id; it('POST 201 + default status todo',async()=>{ const r=await
req(app).post('/tasks').send({title:'Buy milk'});
expect(r.status).toBe(201);expect(r.body.status).toBe('todo');id=r.body.id; }); it('GET
returns array',async()=>expect((await req(app).get('/tasks')).status).toBe(200)); it('GET
?status=todo filters',async()=>{ const r=await req(app).get('/tasks?status=todo');
expect(r.body.every(t=>t.status==='todo')).toBe(true); }); it('GET /:id fetches
task',async()=>{ const r=await req(app).get(`/tasks/${id}`);
expect(r.status).toBe(200);expect(r.body.title).toBe('Buy milk'); }); it('PUT updates
status',async()=>{ const r=await req(app).put(`/tasks/${id}`).send({status:'done'});
expect(r.body.status).toBe('done'); }); it('POST 400 - title missing',async()=>expect((await
req(app).post('/tasks').send({})).status).toBe(400)); it('POST 400 - title>200',async()=>
expect((await req(app).post('/tasks').send({title:'x'.repeat(201)})).status).toBe(400));
it('GET 404 - unknown id',async()=>expect((await
req(app).get('/tasks/none')).status).toBe(404)); it('DELETE 204 + confirms
removal',async()=>{ expect((await req(app).delete(`/tasks/${id}`)).status).toBe(204);
expect((await req(app).get(`/tasks/${id}`)).status).toBe(404); }); });
```

Test	Spec constraint	Result
POST 201 + default status todo	POST returns 201 · default status is todo	PASS ✓
GET /tasks returns array	GET /tasks returns 200 with JSON array	PASS ✓
GET ?status=todo filters	?status= query parameter filters correctly	PASS ✓
GET /tasks/:id fetches task	GET /tasks/:id returns 200 with task body	PASS ✓
PUT /tasks/:id updates status	PUT updates fields and returns 200	PASS ✓
POST 400 — title missing	400 if title is absent from request body	PASS ✓
POST 400 — title > 200 chars	400 if title exceeds 200 character limit	PASS ✓
GET 404 — unknown id	404 if :id does not exist in the database	PASS ✓
DELETE 204 + confirms removal	204 on DELETE · subsequent GET returns 404	PASS ✓

Ch 15

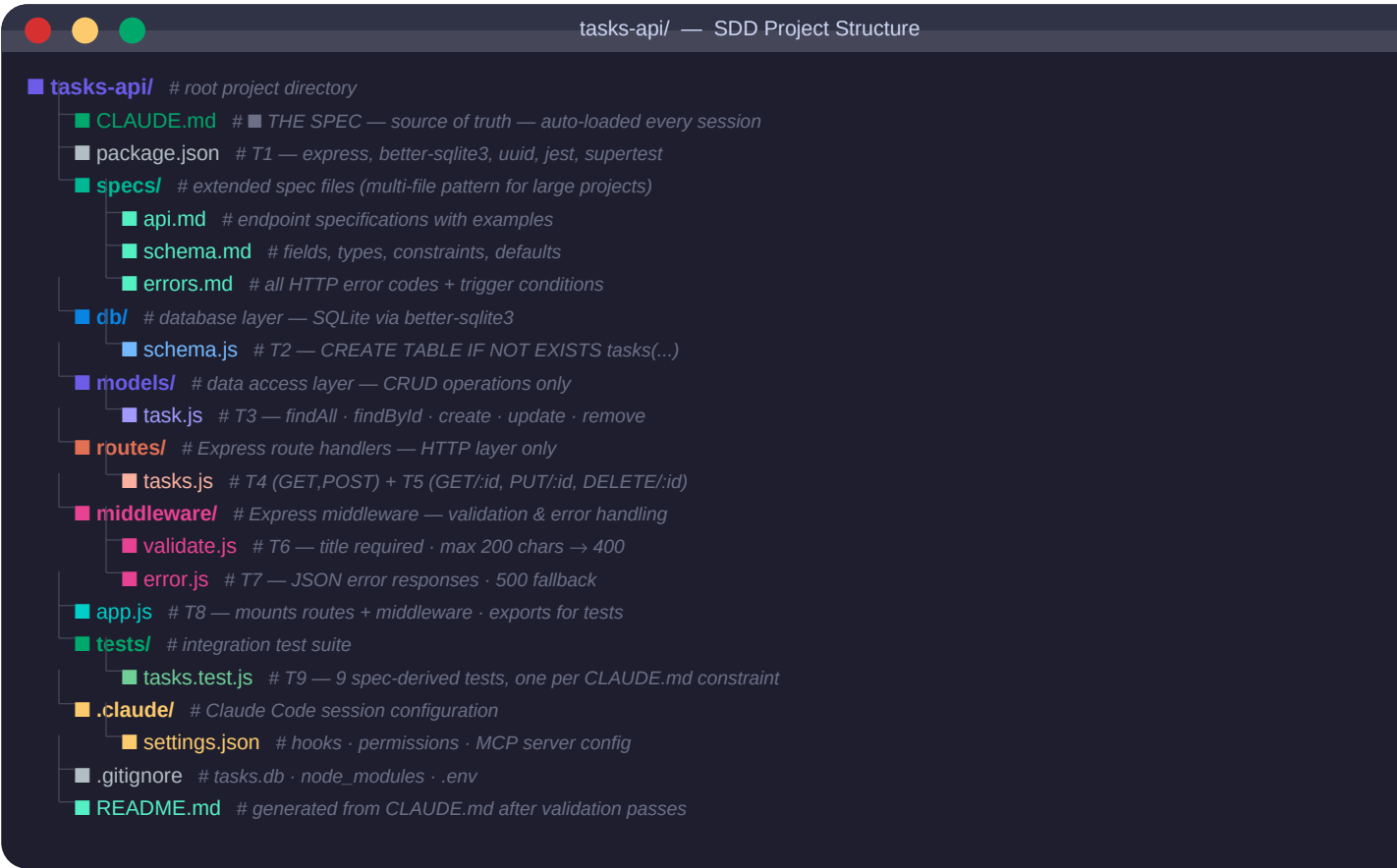
Folder Structure

SDD terminal tree · All files annotated · File creation order

The SDD folder structure mirrors the task list which mirrors the spec. Every file's purpose is immediately obvious from its location. The creation order matches the dependency graph — database before model, model before routes, routes before app.

SDD Project Tree — Annotated Terminal View

Figure 15.1 — Full annotated project tree with task numbers and spec references.



File Creation Order (follows task list dependency graph)

Task	File	Why this order
T1	package.json	Dependencies declared first — all subsequent require() calls depend on this
T2	db/schema.js	Database before model — model requires the db connection
T3	models/task.js	Model before routes — routes require Task model functions
T4	routes/tasks.js (GET,POST)	Routes before app — app mounts the router
T5	routes/tasks.js (GET/:id,PUT,DEL)	Same file, split into two tasks for review
T6	middleware/validate.js	Middleware before app — app.use() requires this

Task	File	Why this order
T7	middleware/error.js	Error handler last in middleware chain
T8	app.js	Entry point last — after all dependencies are defined
T9	tests/tasks.test.js	Tests last — supertest requires app.js to exist

Ch 16

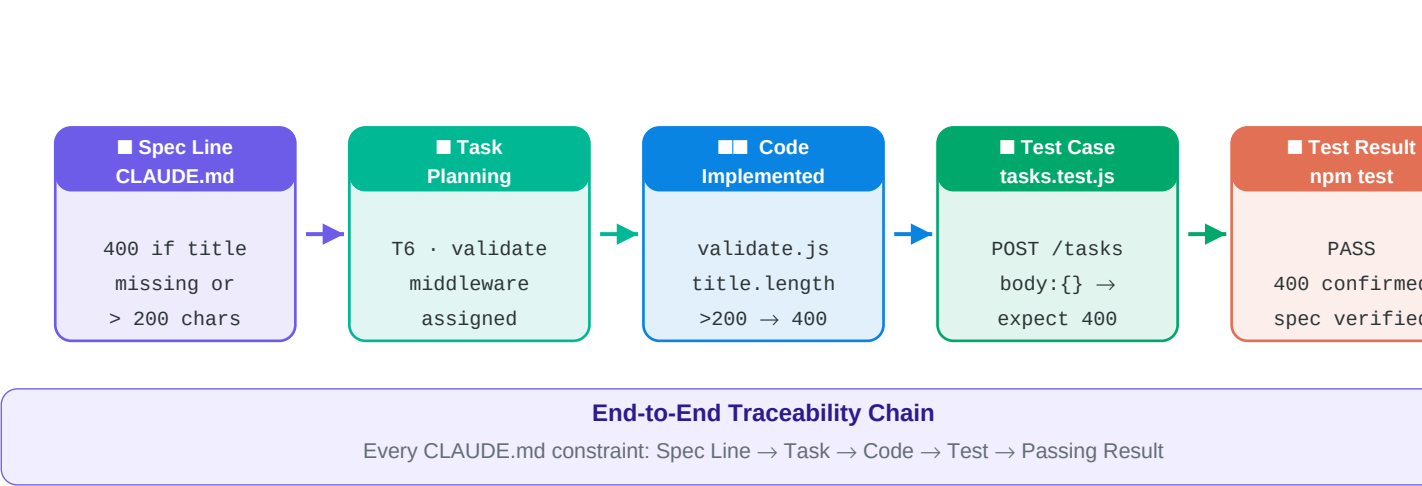
Traceability & Observability

Chain diagram · 5 obs. layers · Full traceability matrix

Traceability is the property that every requirement in CLAUDE.md can be followed through the task list → code → test → passing test result. Observability is the property that every decision and verification in SDD is visible, auditable, and reproducible.

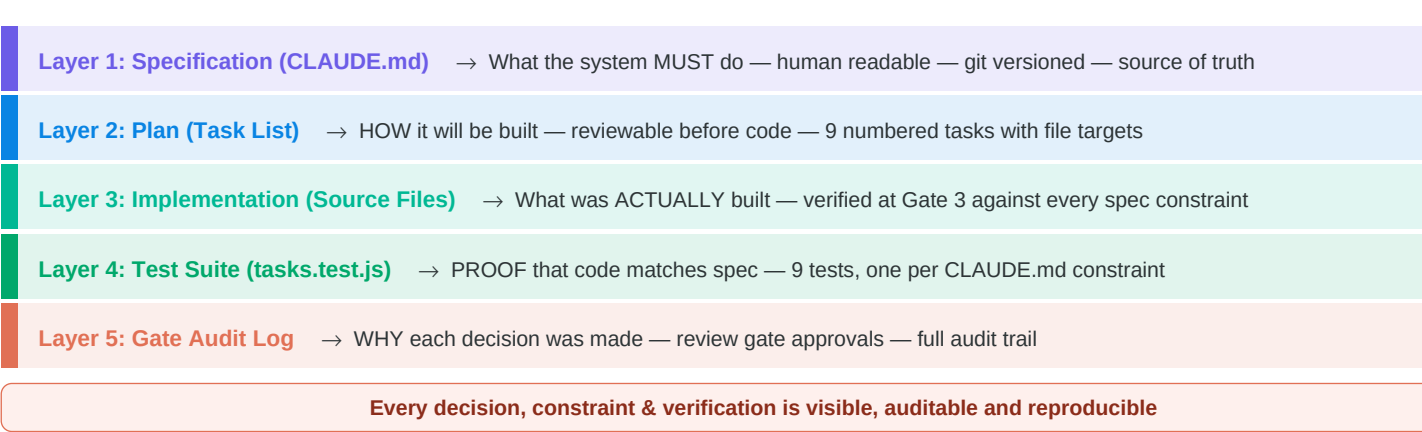
End-to-End Traceability Chain

Figure 16.1 — A single spec constraint traced all the way to a passing test result.



SDD Observability Layers

Figure 16.2 — Five observability layers: specification to gate audit log.



Full Traceability Matrix — Tasks API

CLAUDE.md Constraint	Task	Implementation file	Test case	Status
POST /tasks returns 201	T4	routes/tasks.js L6	POST 201 test	PASS ✓
Default status is 'todo'	T3	models/task.js L7	POST default status	PASS ✓

CLAUDE.md Constraint	Task	Implementation file	Test case	Status
GET /tasks returns JSON array	T4	routes/tasks.js L3	GET array test	PASS ✓
?status= filter works	T4	routes/tasks.js L3	GET filter test	PASS ✓
PUT /tasks/:id updates fields	T5	routes/tasks.js L17	PUT update test	PASS ✓
400 if title missing	T6	middleware/validate.js L3	POST 400 no title	PASS ✓
400 if title > 200 chars	T6	middleware/validate.js L5	POST 400 long title	PASS ✓
404 if :id not found	T4,T5	routes/tasks.js L10,19,24	GET 404 test	PASS ✓
DELETE returns 204	T5	routes/tasks.js L26	DELETE 204 test	PASS ✓
Record gone after DELETE	T5	routes/tasks.js L23	DELETE 404 confirm	PASS ✓
500 on unhandled error	T7	middleware/error.js L3	Error handler test	PASS ✓

What Breaks Traceability?

- **Missing spec line** — a constraint not in CLAUDE.md will not be implemented or tested.
- **Missing task** — a spec line with no task means no file gets written for it.
- **Spec drift** — code patched without updating CLAUDE.md breaks all downstream traces.
- **Test written to pass** — a test shaped around existing code, not a spec constraint.
- **Vague spec** — 'returns an error' instead of '400 with {error: string}' makes tracing impossible.

Ch 17

Comparison Tables & Quick Reference

Master table · CLI prompts · Gates · Anti-patterns · Index

All Five Approaches — Master Comparison

Dimension	Vibe	TDD	BDD	DDD	SDD
Starts with	A prompt	Failing test	Gherkin	Domain model	CLAUDE.md
Design before code?	No	Tests only	Scenarios	Domain model	Full spec
Non-tech stakeholders?	No	No	Yes	Partially	No (readable)
AI-driven?	Partial	No	No	No	Yes — primary
Context persists?	No	Test files	.feature files	Domain model	CLAUDE.md
Review cadence	Every edit	Per cycle	Per scenario	Per iteration	3 gates
Tests from	Code exists	Mental model	Gherkin	Domain	CLAUDE.md spec
Best for	Scripts	Unit logic	Acceptance	Complex domain	AI features

The 4 Essential Claude Code Prompts

```
# Phase 2 – Generate plan (NO CODE)
claude "Read CLAUDE.md and produce a numbered implementation plan. One task per file. No code yet. Stop for my review."
# Phase 3 – Implement in batches with gates
claude "Implement T1 and T2. Stop after each for review."
claude "Implement T3 through T5. Stop for review."
claude "Implement T6, T7, and T8. Show each file when done."
# Phase 4 – Validate against spec
claude "Implement T9 (integration tests). One test per CLAUDE.md constraint. Run npm test. For each failure, cite the exact CLAUDE.md line it tests and fix the code to match the spec."
```

Review Gate Checklist

Gate	After	Key questions	Common catches
Gate 1	Specify	Every endpoint, schema field & error documented? Stack versioning?	Missing 404 rules, vague field types, no library versions
Gate 2	Plan	Every spec line has a task? File order correct?	Missing middleware task, test task missing, wrong dependencies
Gate 3	Implement	Every spec constraint traceable to code? All hooks passing?	Wrong status code, validation in wrong layer, 204 not returned

Spec Anti-patterns — What NOT to Write in CLAUDE.md

Anti-pattern	Why it fails	Correct spec line
'Returns an error on failure'	Ambiguous — which status? which body?	'Returns 400 with {error: string}'
'Validates input'	What rules? What response?	'400 if title missing or over 200 chars'
'Uses a database'	Which? Which library?	'SQLite via better-sqlite3 9.4.3'
'Returns success'	200? 201? 204?	'Returns 201 with full task object as JSON'

Anti-pattern	Why it fails	Correct spec line
'Handles errors'	Which errors? Which codes?	'500 with {error: string} on unhandled exceptions'

Hook Events Quick Reference

Event	Fires	Best use	Example command
PreToolUse	Before every tool call	Log + validate file path	echo '{cmd}' >> audit.log
PostToolUse	After every tool call	ESLint + TypeScript check	eslint {file} --fix --quiet
Stop	Claude finishes	Full test suite	npm test -- --silent
Notification	User input needed	Slack gate notification	curl -X POST \$HOOK -d 'gate'

Complete Terminology Index

Term	Ch	One-line definition
Agent	6, 9	Autonomous AI process executing multi-step tasks with tools
Agentic Loop	6, 7	Plan→execute→observe→revise cycle inside Claude Code
Bash Tool	6, 7	Runs shell commands: npm, git, tests, linters
BDD	Ch 3	Behaviour-Driven Dev — Given/When/Then Gherkin scenarios
CLAUDE.md	6, 11, 13	Auto-loaded spec file — the source of truth for SDD
Context Window	6, 7	Maximum tokens readable in one Claude Code session
DDD	Ch 4	Domain-Driven Design — ubiquitous language, bounded contexts
Edit Tool	6, 7	Reads and writes files on disk — produces all source files
Gate / Review Gate	6, 13	Human checkpoint between SDD phases
Guardrails	6	Rules limiting Claude Code's allowed actions
Hook	6, 8	Shell command auto-fired at PreToolUse/PostToolUse/Stop/Notification
MCP	6, 12	Model Context Protocol — connects Claude to external tools
Memory — In-context	6, 11	Active session data — lost when session ends
Memory — Project	6, 11	CLAUDE.md — persists across sessions in git
Memory — User	6, 11	Developer preferences in claude.ai account
Multi-agent	6, 9	Multiple Claude agents working in parallel on same spec
Orchestrator	6, 9	Top-level agent assigning task groups to sub-agents
Permissions	6	Allowed/blocked actions configured in settings.json
Skill	6, 10	SKILL.md encoding best practices for a specific task type
Spec	5, 6	Written specification in CLAUDE.md — source of truth

Term	Ch	One-line definition
Spec Drift	6	Code diverging from CLAUDE.md — the SDD anti-pattern
SDD	Ch 5	Spec-Driven Development: specify→plan→implement→validate
Sub-agent	6, 9	Parallel agent handling one task group from orchestrator
System Prompt	6, 7	Hidden operator instructions before session starts
TDD	Ch 2	Test-Driven Development — Red-Green-Refactor cycle
Tool Call	6, 7	Claude's request to use edit, bash, or MCP capability
Traceability	6, 16	Spec constraint traceable through to a passing test result
Validate	6, 13, 14	Phase 4 — spec-derived tests all must pass
Vibe Coding	Ch 1	Freestyle prompt-driven development without a written spec

Ch 18

Common Mistakes & How to Fix Them

Anti-patterns · Pitfalls · Corrections with examples

Even experienced developers make avoidable mistakes when adopting SDD. This chapter documents the most common pitfalls, why they happen, what goes wrong, and how to fix them before they become expensive.

Mistake 1 — Spec Too Vague

Anti-pattern: writing ambiguous spec lines that Claude must guess at.

Wrong spec line	What Claude produces	Correct spec line
'Returns an error'	Any status code, any body shape	'400 with {error: string}'
'Validates input'	Any validation logic it invents	'400 if title missing or over 200 chars'
'Uses a database'	Chooses any DB and library	'SQLite via better-sqlite3 9.4.3'
'Returns success'	200, 201, or 204 at random	'Returns 201 with full task object as JSON'
'Handle errors properly'	Whatever Claude considers proper	'500 with {error: string} on unhandled'
'Should be fast'	No measurable constraint	'p95 response time < 200ms'

Mistake 2 — Skipping Review Gates

The most common mistake: approving the task list without reading it, then discovering at Gate 3 that a critical middleware layer was missing. The cost of restructuring a codebase is 10-50x higher than restructuring a task list.

- Gate 2 fix: spend 5 minutes tracing every task back to a spec line. If a spec line has no task, add one.
- Gate 3 fix: open each source file and find the line of code implementing each CLAUDE.md constraint.

Mistake 3 — Patching Code Without Updating the Spec

```
// Developer finds a bug: the 200-char limit should be 300 chars // They fix it directly in
routes/tasks.js: if (title.length > 300) ... // patched! // But CLAUDE.md still says: '400 if
title over 200 chars' // Next SDD session: Claude re-enforces 200 → production regression //
CORRECT PROCESS: // 1. Update CLAUDE.md: '400 if title over 300 chars' // 2. Then update the
code // 3. Then update the test // 4. Run npm test → all green
```

Mistake 4 — Giant Monolithic CLAUDE.md

A CLAUDE.md that is 500+ lines causes two problems: it consumes too much context window, and Claude loses coherence across the spec. Fix: use the multi-file index pattern.

```
# BAD: one 600-line CLAUDE.md with everything # GOOD: root CLAUDE.md as a concise index (< 40
lines) # CLAUDE.md ## Stack: Node.js 20 | Express 4 | SQLite ## Sub-specs: # /specs/api.md –
endpoint definitions # /specs/schema.md – data schema # /specs/errors.md – error codes #
/specs/auth.md – authentication (v2) # /specs/conventions.md – coding standards
```

Mistake 5 — Writing Tests After Shipping

Tests written after implementation validate the code that exists, not the requirements that were defined. This means bugs in the implementation get baked into the tests.

- SDD fix: always write tests in Phase 4 before any manual end-to-end testing.
- Prompt Claude: 'For each test, cite the exact CLAUDE.md line it validates.'
- If Claude cannot cite a CLAUDE.md line for a test, the test is not spec-derived — delete it.

Mistake 6 — Spec Drift Over Time

Scenario	What breaks	Fix
Feature added without spec update	Next session Claude reverts the feature	Update CLAUDE.md before coding
Bug fix changes behaviour silently	Spec says old behaviour; Claude re-introduces the bug	Update spec first, then fix
Team member updates code without spec	Claude sessions from other team members regress	CLAUDE.md is git-tracked — require PR reviews
Spec migrated to Notion but not CLAUDE.md	Claude reads stale CLAUDE.md	Keep CLAUDE.md as the primary spec always

Ch 19

SDD in CI/CD Pipelines

Automating spec validation · GitHub Actions · Quality gates

SDD integrates naturally into CI/CD pipelines. The CLAUDE.md spec becomes the contract that the pipeline enforces automatically — no human needs to be present for routine spec validation. Hooks and automated tests run on every commit.

CI/CD Integration Architecture

CI Stage	What runs	SDD artifact validated
Lint	eslint + prettier	Code style matches conventions in CLAUDE.md
Type check	tsc --noEmit	TypeScript types match spec schema shapes
Unit tests	npm test -- --testPathPattern=unit	Model layer matches spec schema + error rules
Integration tests	npm test -- --testPathPattern=tasks	All 9 spec-derived tests pass
Spec drift check	node scripts/check-spec-drift.js	Every CLAUDE.md endpoint has a route
Contract test	npm test -- --testPathPattern=contract	Response shapes match spec definitions

GitHub Actions Workflow

```
# .github/workflows/sdd-validate.yml
name: SDD Spec Validation
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      # Stage 1: Lint (spec: coding conventions)
      - name: Lint
        run: npx eslint src/ --quiet
      # Stage 2: Integration tests (spec: all constraints)
      - name: Run spec-derived tests
        run: npm test -- --ci --coverage
      # Stage 3: Spec drift check
      - name: Check spec drift
        run: node scripts/check-spec-drift.js
      # Stage 4: Coverage threshold (spec: all constraints tested)
      - name: Coverage gate
        run: |
          COVERAGE=$(cat coverage/coverage-summary.json | jq '.total.lines.pct')
          if (( $(echo "$COVERAGE < 85" | bc -l) )); then
            echo 'Coverage below 85% - spec constraints not fully tested'
            exit 1
          fi
```

Spec Drift Check Script

```
// scripts/check-spec-drift.js
// Parses CLAUDE.md and verifies every endpoint has a matching route
const fs = require('fs');
const path = require('path');
const spec = fs.readFileSync('CLAUDE.md', 'utf8');
const routeFile = fs.readFileSync('routes/tasks.js', 'utf8');
// Extract endpoint lines from spec
const endpointRegex = /^(\GET|\POST|\PUT|\DELETE|\PATCH)\s+(\S+)/gm;
const specEndpoints = [...spec.matchAll(endpointRegex)].map(m => ({ method: m[1].toLowerCase(), path: m[2] }));
let failures = 0;
for (const { method, path: p } of specEndpoints) {
  const routePattern = new RegExp(`router\\.${method}\\((('${p.replace(':id',':[^\\s]+')})')`);
  if (!routePattern.test(routeFile)) {
    console.error(`SPEC DRIFT: ${method.toUpperCase()} ${p} has no route`);
    failures++;
  }
}
if (failures > 0) {
  console.error(`${failures} spec drift(s) detected`);
  process.exit(1);
}
console.log('Spec drift check: PASSED - all endpoints implemented');
```

SDD Branch Strategy

- **spec/feature-name** branch — update CLAUDE.md first. Open a PR for spec review before any code.
- **impl/feature-name** branch — implements the approved spec. PR must include passing CI including spec-drift check.
- **main** branch — CLAUDE.md and code are always in sync. Spec drift check blocks merges.

Golden rule for CI: if the spec drift check fails, the build fails. No exceptions. This enforces the SDD principle that CLAUDE.md is always the source of truth.

Ch 20

Real-World SDD Patterns

Team workflows · Versioning · Extending the API · FastAPI

SDD works at any scale — solo projects to large engineering teams. This chapter covers practical patterns that make SDD sustainable over time.

Pattern 1 — Spec-First Feature Development

Step	Who	What	Artifact
1	Product Manager	Write the feature requirement in plain English	Requirement doc
2	Developer	Translate requirement into CLAUDE.md spec additions	Updated CLAUDE.md
3	Team	Review spec PR — agree on endpoint shapes, error codes	Approved spec PR
4	Developer	Run SDD phases 2-4 with Claude Code	Working implementation
5	QA	Verify spec-derived tests cover all acceptance criteria	Test sign-off
6	CI/CD	Spec drift check + full test suite on every commit	Green pipeline

Pattern 2 — Extending the Tasks API (Adding Auth)

When requirements change, the SDD process ensures clean extension without drift. Here is how to add JWT authentication to the existing Tasks API using SDD.

Step 1 — Update CLAUDE.md first

```
# CLAUDE.md additions for v2 ## Auth (v2 – added 2025-06) All endpoints now require:
Authorization: Bearer 401 if token missing or invalid → {error: 'Unauthorized'} 403 if token
valid but wrong user → {error: 'Forbidden'} ## New endpoint POST /auth/login #
body:{email,password} returns:{token,expiresIn} POST /auth/logout # header: Bearer token
returns: 200 ## Task schema (v2 additions) userId TEXT UUID of the owning user – set on
create, immutable
```

Step 2 — New task list (Plan)

```
# New tasks for auth extension: T10 middleware/auth.js # JWT verify middleware T11
routes/auth.js # POST /auth/login + /auth/logout T12 models/user.js # User model (findByEmail,
create) T13 db/schema.js # Add users table migration T14 tests/auth.test.js # Spec-derived
auth tests
```

Pattern 3 — SDD with Python FastAPI

SDD is framework-agnostic. The same four phases work with any language or framework. Here is the Tasks API CLAUDE.md adapted for Python + FastAPI.

```
# CLAUDE.md – Tasks API (Python/FastAPI version) ## Stack Python 3.12 | FastAPI 0.110 | SQLite
viaaiosqlite 0.20 | pydantic 2.6 ## Endpoints (same as Node.js version) GET /tasks # list;
?status= filter; 200 POST /tasks # body: TaskCreate; 201 GET /tasks/{task_id} # 200; 404 if
not found PUT /tasks/{task_id} # body: TaskUpdate; 200; 404 DELETE /tasks/{task_id} # 204; 404
if not found ## Pydantic schemas class TaskCreate(BaseModel): title: str = Field(...,
```

```
max_length=200) class TaskUpdate(BaseModel): title: Optional[str] = Field(None,
max_length=200) status: Optional[Literal['todo','in_progress','done']] = None class
Task(BaseModel): id: UUID; title: str; status: str; created_at: datetime ## Error rules 422
pydantic validation fails → FastAPI default error body 404 task_id not found → {detail: 'Task
not found'}
```

```
# FastAPI implementation (T4 equivalent) from fastapi import FastAPI, HTTPException from uuid
import uuid4 from datetime import datetime app = FastAPI() @app.post('/tasks',
status_code=201) async def create_task(task: TaskCreate) -> Task: new_task = Task( id=uuid4(),
title=task.title, status='todo', created_at=datetime.utcnow() ) await db.execute( 'INSERT INTO
tasks VALUES(:id,:title,:status,:created_at)', new_task.dict() ) return new_task
@app.get('/tasks/{task_id}') async def get_task(task_id: UUID) -> Task: row = await
db.fetch_one('SELECT * FROM tasks WHERE id=:id', {'id':str(task_id)}) if not row: raise
HTTPException(status_code=404, detail='Task not found') return Task(**row)
```

Pattern 4 — SDD with AWS Bedrock Agent

For teams building agent-based applications on AWS Bedrock, SDD provides the specification layer that keeps multi-model orchestration coherent.

```
# CLAUDE.md — Bedrock Agent SDD spec ## Goal An AWS Bedrock agent that processes customer
support tickets using Claude and routes escalations to human agents via SQS. ## Agent actions
(Bedrock action groups) classify_ticket(ticket_text: str) → {category, confidence, priority}
fetch_customer_history(customer_id: str) → CustomerHistory[] escalate_to_human(ticket_id:
str, reason: str) → {queue_url, message_id} auto_resolve(ticket_id: str, resolution: str) →
{status: 'resolved'} ## Foundation model claude-sonnet-4-20250514 for classification and
response generation ## Error handling ThrottlingException → exponential backoff max 3 retries
ModelErrorException → escalate_to_human with reason='model_error'
```

Ch 21

Practice Exercises

Apply SDD to 5 real scenarios with solution hints

The best way to internalize SDD is to write specs. These five exercises progress from simple to complex. Each includes a scenario, what to write in CLAUDE.md, and what the task list should look like.

Exercise 1 — URL Shortener API (Beginner)

Scenario: Build a URL shortening service. Users submit a long URL and get back a short code. Anyone can use the short code to be redirected to the original URL.

Write the CLAUDE.md spec for:

- POST /shorten — accepts {url: string}, validates it is a real URL, returns {code, shortUrl}
- GET /{code} — redirects (301) to the original URL, 404 if code not found
- GET /stats/{code} — returns {code, originalUrl, clicks, createdAt}

Expected task list (hint):

```
T1: package.json (express, nanoid, better-sqlite3) T2: db/schema.js (urls table: code, originalUrl, clicks, createdAt) T3: models/url.js (create, findByCode, incrementClicks) T4: routes/shorten.js (POST /shorten) T5: routes/redirect.js (GET /:code, GET /stats/:code) T6: middleware/validateUrl.js (isValidUrl check) T7: app.js T8: tests/url.test.js (7 spec-derived tests)
```

Exercise 2 — Notes API with Tags (Intermediate)

Scenario: Build a notes API where each note has a title, body, and an array of tags. Users can filter notes by tag. Tags are stored as a comma-separated string.

Write the CLAUDE.md spec for:

- POST /notes — {title, body, tags: string[]} — returns 201
- GET /notes — list all, with optional ?tag= filter
- PUT /notes/:id — update any combination of title, body, tags
- DELETE /notes/:id — 204
- GET /notes/tags — returns unique array of all tags used

Tricky spec decisions to make:

- How are tags stored? (comma-separated TEXT vs separate tags table)
- What is the 400 rule for missing title vs missing body?
- What does GET /notes/tags return if no notes exist?

Exercise 3 — E-commerce Cart API (Advanced)

Scenario: Build a shopping cart API. Carts belong to users, contain items with quantity and price, and have a calculated total. Items can be added, updated, or removed.

Write the CLAUDE.md spec for:

- POST /carts — create a new empty cart for a user, returns {cartId, userId, items:[], total:0}
- POST /carts/:cartId/items — add item {productId, quantity, price}, returns updated cart

- PUT /carts/:cartId/items/:productId — update quantity (0 = remove)
- GET /carts/:cartId — returns cart with items and calculated total
- DELETE /carts/:cartId — delete cart and all items

Advanced spec decisions:

- How is total calculated? (sum of item.price * item.quantity)
- What happens if quantity is set to 0 via PUT? (remove the item)
- What is the 409 rule? (adding a productId that already exists in the cart)

Exercise 4 — Webhook Delivery System (Expert)

Scenario: Build a webhook delivery system. Applications register webhook endpoints. When events occur, the system delivers them to all registered endpoints with retry logic.

Write the CLAUDE.md spec for:

- POST /webhooks — register {url, events: string[], secret} — returns {webhookId, status:'active'}
- GET /webhooks/:webhookId/deliveries — list delivery attempts with status and response code
- POST /events — trigger an event {type, payload} — fan-out to all matching webhooks
- Delivery retry: 3 attempts with exponential backoff (1s, 5s, 25s)
- Delivery status: pending | delivered | failed

Key spec decisions:

- How are events matched to webhooks? (events array contains the event type)
- What HTTP status on POST /events? (202 Accepted — async processing)
- What constitutes a successful delivery? (webhook endpoint returns 2xx)

Exercise tip: for each exercise, write the full CLAUDE.md spec before looking at the hints. Then compare your spec to the hints. The differences reveal what you need to make more explicit. A good spec leaves nothing for Claude to assume.

Spec Writing Checklist (use for every exercise)

Checklist item	Done?
Every endpoint has a method, path, request body, and response body	■
Every endpoint has explicit success status code (200/201/204)	■
Every error condition has an HTTP code and response body shape	■
Every schema field has type, constraints, and default if applicable	■
Tech stack is specified with exact library versions	■
Coding conventions are documented (naming, SQL style, error shape)	■
Edge cases are covered (empty arrays, zero values, null fields)	■
Could a new developer implement this correctly with only this file?	■

Summary & What's Next

Core principles recap · Future of SDD · Resources

The Core SDD Insight

The fundamental insight of Spec-Driven Development is this: **AI agents are powerful implementers but poor requirement discoverers**. When you ask Claude Code to build something without a spec, it makes dozens of implicit decisions — status codes, field names, validation rules, error shapes. Each implicit decision is a potential disagreement with what you actually wanted. SDD eliminates implicit decisions by making all requirements explicit in CLAUDE.md before any code runs.

The 10 Rules of Spec-Driven Development

#	Rule	Why it matters
1	Write the spec before any code	Implicit decisions cause expensive rewrites
2	Every endpoint, field, and error must be explicit	Vague specs produce wrong code
3	Approve the task list before implementing	Structural problems are cheap to fix in a list
4	Implement in batches with review gates	Catch code-spec gaps early
5	Every test must cite its spec constraint	Tests should prove requirements, not just run
6	Update the spec before changing the code	Spec drift is the #1 cause of SDD failure
7	Use hooks for automated quality gates	Remove manual gate checking overhead
8	Keep CLAUDE.md concise — index sub-specs	Context overflow kills coherence
9	Commit CLAUDE.md changes as separate PRs	Spec changes need explicit review
10	Never vibe code multi-file features	Complexity needs structure

When SDD Shines Most

- **REST APIs** — every endpoint, status code, and error rule maps perfectly to a CLAUDE.md line.
- **Multi-agent systems** — CLAUDE.md is the coordination contract that keeps parallel agents aligned.
- **Team projects** — CLAUDE.md serves as both spec and documentation for the whole team.
- **Regulated systems** — CLAUDE.md + traceability matrix provides audit trail for compliance.
- **Long-lived codebases** — CLAUDE.md evolves with the system and never goes stale.

The Future of SDD

- **Automatic spec generation** — Claude reads existing code and generates CLAUDE.md from it, enabling SDD adoption on legacy projects.
- **Spec-to-contract testing** — CLAUDE.md is automatically parsed to generate Pact contracts for consumer-driven contract testing.

- **Multi-team spec coordination** — different teams maintain their bounded-context specs; an orchestrator verifies interface compatibility.
- **Spec evolution tracking** — git-tracked CLAUDE.md changes generate automated changelogs and migration scripts.
- **IDE integration** — real-time spec drift detection in VS Code: red underline on code that violates CLAUDE.md.

Summary

You now have a complete picture of Spec-Driven Development: from the definition of vibe coding and how it differs from TDD, BDD, and DDD, through all the technical terminology of Claude Code's architecture, to the practical mechanics of hooks, agents, sub-agents, MCP, skills, and memory. You have a working 9-task Tasks API implementation with 11 spec-derived tests, a full traceability matrix, a CI/CD workflow, and five practice exercises to build your SDD skills.

The single most important habit to build: **open CLAUDE.md before you open any source file**. If the change you are about to make is not described in CLAUDE.md, write it there first. That discipline — spec first, always — is what makes SDD work.

Spec-Driven Development with Claude Code — Complete Technical Reference

Author: Jampala Naresh · Software Engineer & Developer Advocate · First Edition · 2025

AI-Assisted Development Handbook Vol. 1 · © 2025 Jampala Naresh · All rights reserved